Техническое задание и исследовательский отчет: Разработка генеративной музыкальной среды в Telegram Mini App

1. Введение и концептуальный обзор

1.1. Контекст и цели проекта

В современной экосистеме цифрового аудио наблюдается отчетливый тренд на демократизацию инструментов создания музыки. Традиционные цифровые рабочие станции (DAW), такие как Ableton Live или Logic Pro, представляют собой мощные, но громоздкие инструменты, требующие значительных вычислительных ресурсов и профессиональных навыков. В то же время, платформа Telegram, трансформировавшаяся из мессенджера в "супер-приложение" (Super App) благодаря внедрению Mini Apps, открывает уникальные возможности для создания легковесных, социально интегрированных творческих инструментов.
Данный документ представляет собой исчерпывающее техническое задание и аналитический отчет по разработке **Telegram MIDI Studio** — автономного веб-приложения внутри Telegram (TMA), предназначенного для генерации музыкальных идей. Проект нацелен на создание симбиоза классических алгоритмов электронной музыки (эмуляция Roland TB-303, драм-машины TR-808/909) и современных интерфейсных решений. Ключевой задачей является реализация сложной логики алгоритмической композиции — от эвклидовых ритмов до вероятностного секвенсинга — в формате интуитивного мобильного интерфейса, с возможностью мгновенного экспорта результатов в формате MIDI для дальнейшей студийной работы.
Уникальность проекта заключается в комбинации специфических алгоритмов: генератора басовых линий, основанного на логике устройства "Sting by Iftah" [2], и гибридного секвенсора, объединяющего сценическую логику ML-185 [3] с пространственной маршрутизацией MDD Snake.[4] Реализация подобной системы в браузере мобильного устройства требует глубокой оптимизации Web Audio API и нестандартных подходов к UX-дизайну.

1.2. Анализ стека технологий и ограничений платформы

Выбор технологического стека обусловлен жесткими требованиями к производительности аудио в реальном времени и ограничениями среды WebView (в которой запускаются Telegram Mini Apps).

Web Audio API и Tone.js Основой звукового движка выбран фреймворк **Tone.js**. Прямое использование Web Audio API требует написания большого объема низкоуровневого кода для планирования событий (scheduling), что повышает риск ошибок синхронизации. Tone.js предоставляет абстракцию Transport, позволяющую точно синхронизировать синтез баса и воспроизведение сэмплов ударных, что критично для музыкального восприятия.[5] Исследования показывают, что Tone.js эффективно использует нативные узлы AudioContext (например, OscillatorNode, GainNode), что обеспечивает приемлемую производительность даже на мобильных устройствах, хотя и требует осторожного управления ресурсами, особенно на iOS устройствах, где действуют жесткие политики автозапуска аудио.[6]

Telegram Mini Apps SDK Интеграция с Telegram осуществляется через библиотеку @telegram-apps/sdk или стандартный объект window.Telegram.WebApp. Ключевой особенностью разработки под эту платформу является необходимость адаптации под тему пользователя (светлая/темная) в реальном времени и управление состоянием "Главной кнопки" (MainButton) для ключевых действий, таких как экспорт.[8] Важным аспектом является преодоление ограничений iOS WebView на скачивание файлов: стандартный HTML-тег <a download> часто блокируется, что требует реализации альтернативного механизма доставки контента через чат-бота.[9]

React и управление состоянием

Для построения пользовательского интерфейса рекомендуется использовать **React**. Его компонентная модель идеально ложится на структуру музыкального интерфейса (компоненты "Knob", "Fader", "Step Sequencer"). Учитывая высокую частоту обновлений интерфейса (например, движение плейхеда по сетке секвенсора 16 раз за такт), критически важно использовать эффективный менеджер состояния, такой как **Zustand** или **Recoil**, избегая лишних перерисовок (re-renders), которые могут вызвать визуальные лаги и, как следствие, звуковые артефакты (glitches).

---

2. Архитектура звукового движка: Синтез и Сэмплирование

2.1. Эмуляция Roland TB-303 (Acid Bass)

Центральным элементом приложения является генератор басовых линий, эмулирующий культовый синтезатор Roland TB-303. Звучание "Acid" характеризуется специфическим взаимодействием осциллятора, фильтра и огибающих, а также уникальной реализацией портаменто (слайдов) и акцентов.

2.1.1. Тракт синтеза на базе Tone.js

Для достоверной эмуляции необходимо построить цепочку обработки, выходящую за рамки стандартного Tone.MonoSynth.

Осциллятор (VCO)

Оригинальный TB-303 предлагает выбор между пилообразной (Sawtooth) и прямоугольной (Square) волной. Однако прямоугольная волна 303-го не является идеальным меандром; она имеет специфические искажения. В Tone.js рекомендуется использовать Tone.PulseOscillator с модулируемой шириной импульса или кастомный PeriodicWave, если требуется максимальная аутентичность.

Фильтр (VCF) Сердце звука TB-303 — это диодный лестничный фильтр (diode ladder filter) с крутизной среза 18 дБ/октаву (хотя часто моделируется как 24 дБ/октаву в цифровых эмуляциях для упрощения). В Tone.js следует использовать Tone.Filter с типом lowpass и высоким значением резонанса (Q). Критически важным является управление частотой среза через огибающую (Filter Envelope).[11]

- *Modulation Depth (Env Mod):* Параметр, определяющий, насколько сильно огибающая влияет на частоту среза.
- *Decay:* Время спада огибающей фильтра. В TB-303 акцент (Accent) динамически изменяет время Decay и глубину модуляции.

Цепь искажения (Distortion) Чистый звук TB-303 редко используется в жанре Acid Techno. Характерное "рычание" достигается за счет перегрузки. В цепочку эффектов необходимо включить Tone.Distortion или Tone.Chebyshev, чтобы имитировать насыщение аналогового тракта.[12]

2.1.2. Реализация логики "Glide" (Portamento)

Эффект скольжения (Slide) в TB-303 уникален тем, что он происходит с постоянным временем (constant time), в отличие от постоянной скорости (constant rate) в других синтезаторах.[13] В Tone.js параметр portamento в MonoSynth позволяет настроить время скольжения. Однако, программная логика должна динамически включать и выключать портаменто на каждом шаге секвенсора.

- *Алгоритм:* Если текущий шаг имеет флаг slide = true, то при переходе к следующему шагу команда triggerAttack не должна посылаться повторно; вместо этого используется setNote для изменения высоты тона, что заставляет осциллятор плавно "доехать" до новой частоты без перезапуска огибающих (Legato).[14]

2.2. Драм-машина: TR-808 и TR-909

Ритмическая секция базируется на сэмплах легендарных драм-машин Roland.

2.2.1. Управление сэмплами и буферизация

Для реализации драм-машины целесообразно использовать Tone.Sampler или Tone.Players. Учитывая мобильный контекст и ограничения по трафику, необходимо найти баланс между качеством аудио и скоростью загрузки.

- **Формат:** Использование сжатых форматов (MP3 320kbps или AAC) предпочтительнее WAV для Web-среды.
- **Preloading:** Все сэмплы должны быть загружены в буфер (AudioBuffer) при инициализации приложения ("Launch Studio"), до начала воспроизведения. Tone.js предоставляет утилиту Tone.loaded(), возвращающую Promise, который

разрешается после загрузки всех ресурсов. Это предотвращает рассинхронизацию при первом ударе.[5]

2.2.2. Параметры обработки ударных

Для каждого инструмента (Kick, Snare, Hi-Hats) необходимо предусмотреть индивидуальные параметры:

- **Pitch/Tuning:** Изменение скорости воспроизведения сэмпла (playbackRate).
- **Decay:** Управление огибающей амплитуды (особенно для Kick 808, чтобы получить длинный "Bass Boom").
- **Output Routing:** Возможность раздельной обработки (например, добавление реверберации только на Snare).

---

3. Алгоритмическая композиция: Логика генерации

Раздел описывает математические модели, лежащие в основе генеративных функций приложения. Эти алгоритмы отвечают за создание музыкального контента без прямого ввода нот пользователем.

3.1. Генератор баса: Логика "Sting by Iftah"

Референсное устройство "Sting" для Ableton Live [2] использует уникальный подход к генерации кислотных линий, основанный на морфинге параметров. Техническое задание требует реализации двух ключевых мета-параметров: **Density** (Плотность) и **Type** (Тип).

3.1.1. Параметр Density (Плотность)

Этот параметр управляет ритмической насыщенностью паттерна. В отличие от простого рандомайзера, который случайным образом включает шаги, алгоритм "Sting" использует концепцию "раскрытия" последовательности.

- **Реализация:**
  1. Генерируется "Seed" (зерно) — последовательность из 16 случайных значений веса (от 0.0 до 1.0) для каждого шага.
  2. Параметр Density (значение от 0.0 до 1.0) выступает в роли порога (Threshold).
  3. Шаг i становится активным (Note On), если $Weight_i < Density$.
  - *Результат:* При увеличении Density новые ноты добавляются, а старые остаются на местах. Это создает ощущение музыкального развития, а не хаотичной смены ритма при повороте ручки.

3.1.2. Параметр Type (Тональность -> Атональность)

Этот параметр управляет мелодическим содержанием и вероятностным распределением высоты тона.

- **Реализация:**

Параметр Type интерполирует выбор нот между несколькими наборами правил (Scale Constraints):

- **0% - 30% (Strict Tonal):** Ноты квантуются строго по ступеням трезвучия (Root, 3rd, 5th) выбранной тональности. Вероятность появления тоники — максимальная.
- **30% - 70% (Diatonic):** Используются все ступени выбранного лада (например, фригийский или минорная пентатоника).
- **70% - 100% (Acid/Atonal):** Вводятся хроматизмы (Blue notes), тритоны и случайные скачки на октаву. Алгоритм начинает игнорировать ладовое тяготение, имитируя "ошибки" или случайный перебор частот, характерный для жанра IDM.[2]

3.1.3. Генерация Акцентов и Слайдов

Для создания аутентичного рисунка 303, атрибуты Accent и Slide также должны генерироваться вероятностно.

- **Правило связности слайдов:** Слайд (Slide) на шаге N имеет музыкальный смысл только если шаг $N+1$ активен. Генератор должен проверять состояние следующего шага перед назначением атрибута Slide. Если $N+1$ неактивен, вероятность слайда принудительно обнуляется или (в альтернативной реализации) шаг $N+1$ принудительно активируется (Tie).

3.2. Эвклидовы ритмы (Алгоритм Бьоркланда)

Секция ударных инструментов использует принципы эвклидовой геометрии для распределения ударов. Алгоритм, описанный Годфридом Туссеном [17], позволяет генерировать ритмы, наиболее равномерно распределяя k импульсов (pulses) на n шагах (steps).

3.2.1. Математическая модель

Алгоритм Бьоркланда (Bjorklund's algorithm) работает по принципу рекурсивного разделения остатка.

Пример: Распределить 5 ударов на 13 шагов ($E(5, 13)$).

1. Начальное состояние: 5 групп и 8 групп.
   ``
2. Паруем единицы с нулями (останавливаемся, когда заканчивается меньшая группа):
   `` (остаток три нуля)
3. Паруем остаток с новыми группами:
   ``
4. Продолжаем рекурсию до финала: 1001010010100.

В коде JavaScript это реализуется через рекурсивную функцию, возвращающую

бинарный массив (Pattern Array). Библиотеки типа euclidean-rhythms [17] уже содержат оптимизированные реализации этого алгоритма.

3.2.2. Полиритмия и полиметрия

Для каждого инструмента (Kick, Snare, Hi-Hat) пользователь задает свои значения k и n.

- Если Snare имеет длину цикла $n = 16$, а Hi-Hat $n = 15$, их паттерны будут смещаться друг относительно друга на 1 шаг каждый такт, создавая длинный, развивающийся полиметрический цикл (phasing), характерный для техно-музыки.[19] Приложение должно визуализировать это смещение в интерфейсе.

---

4. Спецификация секвенсора: ML-185 и MDD Snake

Данный модуль объединяет две различные парадигмы секвенсирования: линейно-сценическую (ML-185) и пространственно-координатную (MDD Snake).

4.1. Логика ML-185: Сценический секвенсор

В отличие от традиционного пошагового секвенсора, где каждый шаг имеет фиксированную длительность (например, 1/16), ML-185 оперирует понятием **Стадия (Stage)**.[3]

4.1.1. Параметры Стадии

Секвенсор состоит из 8 стадий. Каждая стадия имеет настройки:

Параметр	Описание	Техническая реализация
Pitch	Высота тона	MIDI Note Number (0-127). Значение берется из MDD Snake (см. ниже).
Velocity	Громкость	Коэффициент усиления (0.0 - 1.0).
Length	Длительность	Количество тактовых импульсов (Clock Pulses), которое длится стадия (1-8).
Mode	Режим гейта	Логика срабатывания огибающей (см. ниже).
Count	Повторы	Количество повторений стадии перед переходом к следующей.

4.1.2. Режимы Гейта (Gate Modes)

Реализация режимов гейта требует сложной логики в обработчике событий Tone.Transport:

1. **Mode 0 (Mute):** Стадия длится заданное время, но нота не звучит (молчание/пауза).
2. **Mode 1 (Single):** Гейт открывается один раз в начале стадии и удерживается до её конца (длинная нота).
3. **Mode 2 (Multi/Retrigger):** Гейт перезапускается на каждом тактовом импульсе внутри стадии (эффект "пулемета" или Ratcheting).
4. **Mode 3 (Hold/Tie):** Гейт не закрывается при переходе от *предыдущей* стадии и не закрывается в конце текущей (связное легато).

4.2. Логика MDD Snake: Пространственный выбор нот

MDD Snake [4] представляет собой матрицу нот (обычно 4x4 или 8x8), по которой движется "голова змеи".

4.2.1. Интеграция с ML-185

В предлагаемой гибридной системе:

- **Ритм (Rhythm/Timing):** Определяется секвенсором ML-185. Он решает, *когда* происходит событие и *как долго* оно длится.
- **Мелодия (Pitch):** Определяется сеткой MDD Snake. При каждом *новом* срабатывании гейта ML-185, позиция в сетке Snake сдвигается на один шаг согласно выбранному паттерну движения.

4.2.2. Паттерны движения (Snake Patterns)

Необходимо реализовать алгоритмы обхода матрицы 4x4:

- *Linear:* -> -> -> -> ...
- *Zig-Zag:* -> ... -> -> -> ... (как змейка).
- *Spiral:* Движение по спирали от внешнего края к центру.
- *Random:* Случайный выбор следующей ячейки (Броуновское движение).

Такой подход позволяет создавать сложные полиритмические мелодии: если длина цикла ML-185 (сумма всех шагов) не совпадает с количеством ячеек в Snake (16), мелодия будет постоянно смещаться относительно сильной доли такта.

---

5. Генерация и Экспорт MIDI файлов

Ключевой функцией приложения является возможность забрать созданную музыку для дальнейшей работы в DAW.

5.1. Выбор библиотеки: midi-writer-js

Для генерации бинарных файлов .mid в браузере оптимальным выбором является библиотека midi-writer-js.[22] Она превосходит jsmidgen [23] благодаря поддержке

множественных треков (Multi-track) и удобному API для добавления метаданных (названия треков, темп).

5.2. Алгоритм конвертации состояния в MIDI

Процесс экспорта представляет собой "рендеринг" данных из стейта приложения в последовательность MIDI-событий. Поскольку генеративные алгоритмы (Sting, Euclidean) могут создавать бесконечную музыку, экспорт должен быть ограничен фиксированным количеством тактов (например, 8 или 16 тактов).

Процедура экспорта:

1. **Симуляция (Dry Run):** Запускается виртуальный цикл генерации на N тактов без воспроизведения аудио.
2. **Запись событий:**
   - *Bass Track (Channel 1):* Каждое событие NoteOn/NoteOff от алгоритма Sting записывается как NoteEvent. Слайды конвертируются в перекрывающиеся ноты (overlapping notes) или ноты полной длительности, чтобы DAW интерпретировала их как легато. Акценты кодируются через Velocity (например, Normal=90, Accent=127).
   - *Drum Track (Channel 10 - GM Standard):* События эвклидовых генераторов маппятся на стандартные ноты General MIDI: Kick -> C1 (36), Snare -> D1 (38), Closed Hat -> F#1 (42).
3. **Сборка файла:** Создается объект Writer, объединяющий треки. Метод buildFile() генерирует Uint8Array байт-код.

---

6. Интеграция с Telegram API: Проблема передачи файлов

Одной из самых сложных технических задач является доставка сгенерированного файла пользователю.

6.1. Ограничения iOS WebView

На устройствах iOS внутри WebView Telegram (и других in-app браузеров) существует давняя проблема с обработкой Blob-ссылок и атрибута download. Прямая попытка программно кликнуть по ссылке <a href="blob:..." download="beat.mid"> часто игнорируется системой безопасности или не приводит к сохранению файла в файловую систему.[9]

6.2. Решение: Передача через Bot API

Единственным надежным способом доставки файла является его отправка ботом в чат с пользователем.

Схема взаимодействия (Client-Server Flow):

1. **Клиент (Mini App):**

    - Генерирует MIDI-файл в формате Base64.
    - Формирует POST запрос к бэкенд-серверу (Middleware).
    - В тело запроса включает: строку Base64, метаданные файла и initData (строка инициализации Telegram для валидации пользователя).[25]
2. **Сервер (Middleware Bot):**
    - Валидирует initData с помощью токена бота (проверка подписи HMAC), чтобы убедиться, что запрос пришел от авторизованного пользователя.
    - Декодирует Base64 в бинарный буфер (Buffer).
    - Использует метод Telegram Bot API sendDocument.[26]
    - *Параметры:* chat_id (извлекается из initData), document (Buffer), caption ("Ваш MIDI файл готов!").
3. **Результат:** Пользователь получает сообщение с файлом прямо в чате, откуда может сохранить его ("Save to Files" на iOS) или переслать на десктоп ("Saved Messages").

6.3. Работа с большими файлами (Optional)

Хотя MIDI-файлы миниатюрны (килобайты), при будущем расширении функционала до экспорта Аудио (WAV/MP3), размер может превысить лимит в 50 МБ для ботов. В таком случае потребуется использование локального сервера Bot API (Local Bot API Server) [27] или разбиение файла на части, но для MIDI текущего решения достаточно.

---

7. UI/UX Дизайн в стиле Telegram

7.1. Дизайн-система и нативность

Интерфейс должен строго следовать гайдлайнам Telegram для обеспечения бесшовного опыта.

- **Цветовая палитра:** Использование CSS-переменных, предоставляемых клиентом Telegram (var(--tg-theme-bg-color), var(--tg-theme-button-color) и т.д.). Это обеспечивает автоматическую поддержку темной и светлой тем.[28]
- **Типографика:** Использование системных шрифтов (San Francisco на iOS, Roboto на Android) через стандартные алиасы.

7.2. Навигация и компоновка

Учитывая сложность управления (три инструмента, секвенсор, настройки), одностраничный интерфейс будет перегружен.

- **Tab Bar:** Рекомендуется использовать навигацию по вкладкам (Tabs) в верхней или нижней части экрана: | | |.
- **Main Button:** Использовать нативную "Главную кнопку" Telegram (внизу экрана) для контекстных действий. На вкладке Export она должна отображать "Отправить в чат", на других вкладках может быть скрыта или отвечать за "Play/Stop".

7.3. Мобильные паттерны для секвенсоров

Отображение 16 или 32 шагов на узком экране смартфона — классическая проблема UX.[29]

- **Горизонтальный скролл:** Является анти-паттерном для музыкальных тактов, так как пользователь теряет контекст всей фразы.
- **Решение "Wrap" (Перенос):** Разбиение 16 шагов на две строки по 8 шагов (2x8 Grid). Это позволяет видеть весь такт целиком без скролла.
- **Touch Targets:** Элементы управления (ручки, фейдеры) должны поддерживать жесты. Для поворотных регуляторов (Knobs) необходимо реализовать логику относительного вертикального перетаскивания (drag up/down), так как вращательные движения пальцем неудобны на маленьких экранах.

---

8. Технический план реализации (Roadmap)

Этап 1: Инициализация и Core Audio (Недели 1-2)

- Настройка репозитория (React + Vite + TypeScript).
- Подключение @telegram-apps/sdk.
- Настройка Tone.Context и обход политик автоплея (Start Audio Button).
- Реализация базового транспорта (Play/Pause, BPM).

Этап 2: Генеративные модули (Недели 3-4)

- Разработка класса AcidGenerator (логика Sting).
- Разработка класса EuclideanGenerator (алгоритм Бьоркланда).
- Создание звукового графа Tone.js (Synth chain, Drum samplers).

Этап 3: Секвенсор и UI (Недели 5-6)

- Верстка компонентов в стиле Telegram.
- Реализация логики ML-185 (Stage controller).
- Визуализация MDD Snake и интерактивная сетка.

Этап 4: Экспорт и Бэкенд (Недели 7)

- Интеграция midi-writer-js.
- Разработка Node.js бота (Express + Telegraf).
- Реализация эндпоинта /upload и связка с Bot API.

Этап 5: Оптимизация и тестирование (Неделя 8)

- Профилирование памяти (Heap snapshots).
- Тестирование на реальных устройствах (iPhone SE, Android Mid-range).
- Тюнинг аудио-параметров (Lookahead, LatencyHint) для устранения треска.

9. Заключение

Предлагаемое техническое задание описывает амбициозный, но реализуемый проект. Telegram MIDI Studio не просто копирует функционал настольных программ, но адаптирует его под парадигму быстрого мобильного творчества ("on-the-go creation"). Использование генеративных алгоритмов компенсирует неудобство ввода нот на тачскрине, превращая пользователя из "наборщика нот" в "куратора процесса". Успешная реализация этого проекта требует высокой квалификации в области Web Audio и глубокого понимания специфики платформы Telegram.

Приложения и Справочные Таблицы

Таблица 1. Сравнение параметров ML-185

Данная таблица необходима для корректной реализации UI контролов секвенсора.

Параметр	Диапазон	Описание функции	Влияние на генерацию
Stage Length	1 - 8	Длительность стадии в шагах (1/16).	Определяет полиметрическую структуру.
Pulse Count	1 - 8	Количество повторов стадии.	Создает репетативные микро-петли.
Gate Mode	0 - 3	Off / Single / Multi / Hold.	Формирует ритмический рисунок (артикуляцию).
Probability	0 - 100%	Вероятность проигрывания стадии.	Добавляет вариативность (Humanization).

Таблица 2. Маппинг параметров "Sting" (303)

Логика интерполяции для генератора баса.

Значение ручки "Type"	Шкала (Scale)	Вес Тоники	Вес Тритона	Описание стиля
0.0 - 0.3	Minor Pentatonic	80%	0%	Стабильный, мелодичный бас.
0.3 - 0.7	Dorian / Phrygian	50%	10%	Техно, хаус.

				Больше движения.
0.7 - 1.0	Chromatic	20%	40%	IDM, Acid. Диссонансы и хаос.

Конец отчета

Источники

1. STING by IFTAH - Discontinued version 1.8 by iftah on maxforlive.com - Max for Live, дата последнего обращения: января 26, 2026, https://maxforlive.com/library/device/8357/sting-by-iftah-a-really-nice-acid-line-generator
2. Expert Mixdown & Mastering Blog | AudioServices Studio - Pheek's, дата последнего обращения: января 26, 2026, https://audioservices.studio/category/blog
3. MDD SnAkE 3.2.3: A Comprehensive Guide to this Max4Live MIDI Sequencer Device, дата последнего обращения: января 26, 2026, https://lame.buanzo.org/max4live_blog/mdd-snake-323-a-comprehensive-guide-to-this-max4live-midi-sequencer-device.html
4. Tone.js, дата последнего обращения: января 26, 2026, https://tonejs.github.io/
5. [AskJS] I can't play multiple sounds on iOS due to restrictions. Work-arounds for a game? : r/javascript - Reddit, дата последнего обращения: января 26, 2026, https://www.reddit.com/r/javascript/comments/1gefdd8/askjs_i_cant_play_multiple_sounds_on_ios_due_to/
6. Mobile performance optimisation/limits? · Issue #188 · Tonejs/Tone.js - GitHub, дата последнего обращения: января 26, 2026, https://github.com/Tonejs/Tone.js/issues/188
7. Mini Apps on Telegram, дата последнего обращения: января 26, 2026, https://core.telegram.org/api/bots/webapps
8. Telegram criticizes Apple for 'intentionally crippling' web app features on iOS - The Register, дата последнего обращения: января 26, 2026, https://www.theregister.com/2022/06/14/telegram_safari/
9. Tone.MonoSynth, дата последнего обращения: января 26, 2026, https://tonejs.github.io/docs/r13/MonoSynth
10. Making distorted 303 acid lines in Logic Pro X using AudioRealism Bassline for psytrance, дата последнего обращения: января 26, 2026, https://www.youtube.com/watch?v=C0iDxU1r3tg
11. Implementing glide or portamento - DSP and Plugin Development Forum - KVR Audio, дата последнего обращения: января 26, 2026, https://www.kvraudio.com/forum/viewtopic.php?t=165353
12. How to implement portamento/legato in a synthdef - Questions - scsynth, дата последнего обращения: января 26, 2026, https://scsynth.org/t/how-to-implement-portamento-legato-in-a-synthdef/5717

13. Acid 4 Live: STING by IFTAH generates acid lines for Ableton Live for free, дата последнего обращения: января 26, 2026, https://cdm.link/acid-4-live-sting-by-iftah-generates-acid-lines-for-ableton-live-for-free/
14. schollz/acid-test: norns script to explore acid basslines through markov chains. - GitHub, дата последнего обращения: января 26, 2026, https://github.com/schollz/acid-test
15. euclidean-rhythms - NPM, дата последнего обращения: января 26, 2026, https://www.npmjs.com/package/euclidean-rhythms
16. dbkaplun/euclidean-rhythm: Euclidean rhythms in JS - GitHub, дата последнего обращения: января 26, 2026, https://github.com/dbkaplun/euclidean-rhythm
17. More than you ever wanted to know about Euclidean Rhythms. You don't need a special plugin, synth, or module to use them - you just need to know how they work. Explanations and ready-to-use examples : r/synthesizers - Reddit, дата последнего обращения: января 26, 2026, https://www.reddit.com/r/synthesizers/comments/yc1rzl/more_than_you_ever_wanted_to_know_about_euclidean/
18. M-185 SEQUENCER - MANUAL OF CONTROLS - Electro-Music.com, дата последнего обращения: января 26, 2026, https://electro-music.com/forum/download.php?id=14034
19. Make Noise Rene - Clockface Modular, дата последнего обращения: января 26, 2026, https://clockfacemodular.com/en/products/make-noise-rene
20. grimmdude/MidiWriterJS: A JavaScript library which provides an API for programmatically generating and creating expressive multi-track MIDI files and JSON. - GitHub, дата последнего обращения: января 26, 2026, https://github.com/grimmdude/MidiWriterJS
21. jsmidgen - NPM, дата последнего обращения: января 26, 2026, https://www.npmjs.com/package/jsmidgen
22. Telegram founder criticizes Apple for intentionally limiting web app features in iOS : r/iphone, дата последнего обращения: января 26, 2026, https://www.reddit.com/r/iphone/comments/vbt588/telegram_founder_criticizes_apple_for/
23. Build a Telegram Mini App Using Pinata's Files API - No Database!, дата последнего обращения: января 26, 2026, https://pinata.cloud/blog/build-a-telegram-mini-app-using-pinatas-files-api-no-database/
24. telegraf - NPM, дата последнего обращения: января 26, 2026, https://www.npmjs.com/package/telegraf
25. How to send large file with Telegram Bot API? - Stack Overflow, дата последнего обращения: января 26, 2026, https://stackoverflow.com/questions/52288231/how-to-send-large-file-with-telegram-bot-api
26. UI Components Library for Telegram Mini Apps · Issue #364 · ton-society/grants-and-bounties - GitHub, дата последнего обращения: января 26, 2026, https://github.com/ton-society/grants-and-bounties/issues/364

27. Step left design pattern for large number of steps on mobile - UX Stack Exchange, дата последнего обращения: января 26, 2026, https://ux.stackexchange.com/questions/124959/step-left-design-pattern-for-large-number-of-steps-on-mobile
28. Horizontal Scroll in App Design, Why Is It Worthy of Attention? | by Trista liu - Medium, дата последнего обращения: января 26, 2026, https://medium.com/@tristaljing/horizontal-scroll-in-app-design-why-is-it-worthy-of-attention-989b363f19b9

Вот серия из 20 последовательных промтов, разработанных для того, чтобы шаг за шагом провести AI-ассистента (или разработчика) от пустой папки до готового Telegram Mini App.

Эти промты составлены с учетом технического отчета (использование Tone.js, React, логики Sting и ML-185). Копируйте и отправляйте их по одному, проверяя результат после каждого шага.

Этап 1: Фундамент и Звуковой Движок (Frontend)

Промт 1: Инициализация проекта

> Создай структуру проекта для Telegram Mini App, используя **React**, **Vite** и **TypeScript**. Установи зависимости: `tone` (Tone.js) для звука, `@telegram-apps/sdk` для интеграции с Telegram, `zustand` для управления состоянием и `lucide-react` для иконок. Настрой базовый манифест и `vite.config.ts`, добавив алиасы для путей (например, `@/components`). Сделай базовый layout, который считывает цветовую схему из `window.Telegram.WebApp.themeParams` и применяет её как CSS-переменные для фона и текста.

Промт 2: Обертка Аудио-контекста

> Напиши React-хук `useAudioEngine`, который инициализирует `Tone.Context`. Реализуй функцию `startAudio`, которая снимает блокировку AudioContext (важно для iOS) и запускает `Tone.Transport`. Добавь большую кнопку "Start App" на главный экран, которая

исчезает после успешного запуска аудио. Реализуй базовый BPM-контроллер, меняющий Tone.Transport.bpm.

Промт 3: Синтезатор TB-303 (Acid Bass)

Создай класс или хук AcidSynth, используя Tone.js. Он должен состоять из Tone.MonoSynthс пилообразной волной, подключенного к Tone.Filter (Lowpass) и Tone.Distortion. Реализуй методы для управления параметрами: Cutoff, Resonance, Envelope Modulation и Decay. Важно: добавь метод triggerNote(note, duration, time, velocity), который умеет обрабатывать акценты (изменяя громкость и открытие фильтра).

Промт 4: Драм-машина (808/909)

Реализуй модуль DrumMachine. Используй Tone.Sampler или Tone.Players. Сделай так, чтобы можно было загружать два набора сэмплов: "808" и "909" (Kick, Snare, HiHat Closed, HiHat Open). Реализуй функцию переключения кита (Kit Selector). Все сэмплы должны проходить через мастер-компрессор драм-секции.

Этап 2: Логика Генерации (Algorithms)

Промт 5: Алгоритм Эвклидовых ритмов

Напиши чистую функцию (pure function) на TypeScript для генерации Эвклидовых ритмов (алгоритм Бьоркланда). Функция должна принимать steps (всего шагов) и pulses(количество ударов) и возвращать массив бинарных значений [1, 0, 1,...]. Также добавь параметр rotate для циклического сдвига массива.

Промт 6: Логика генератора баса "Sting"

Реализуй логику генерации баса, основанную на принципе устройства "Sting". Создай функцию generateBassPattern(density, type, scale, rootNote).

- Density (0-1): определяет вероятность появления нот в 16-шаговом паттерне.
- Type (0-1): определяет мелодику. При 0 — только тоника и квинты, при 0.5 — диатоника, при 1.0 — хроматика и атональные скачки (Acid).
  Функция должна возвращать массив объектов: { note, velocity,

slide, octaveShift }.

Промт 7: Логика ML-185 (Stages)

Создай модель данных для секвенсора типа ML-185. Нам нужна структура из 8 "Стадий" (Stages). Каждая стадия должна иметь параметры: pitch (0-127), velocity, length (длительность в 1/16 долях), pulseCount (сколько раз повторять стадию) и gateMode (0: Mute, 1: Single, 2: Multi, 3: Tie). Напиши функцию, которая преобразует этот список стадий в линейную последовательность событий для Tone.js.

Промт 8: Логика MDD Snake (Grid)

Реализуй логику движения по сетке 4x4 (MDD Snake). Создай класс GridWalker, который хранит текущие координаты (x, y). Реализуй алгоритмы движения: Linear (слева направо), ZigZag, Spiral и Random. Метод next() должен возвращать индекс текущей ячейки (0-15) и вычислять следующую позицию согласно выбранному алгоритму.

Этап 3: Связывание и Состояние (Glue)

Промт 9: Глобальный Store (Zustand)

Настрой глобальное хранилище данных с помощью **Zustand**. Создай слайсы (slices) для:

1. TransportStore (playing, bpm, swing).
2. DrumStore (параметры steps/pulses для каждого инструмента).
3. BassStore (параметры Density, Type, текущий сгенерированный паттерн).
4. SequencerStore (настройки стадий ML-185 и позиция Snake).

Промт 10: Секвенсор (Tone.Sequence)

Создай главный компонент SequencerLoop, который использует Tone.Transport.scheduleRepeat. Внутри колбэка:

1. Вычисляй текущий шаг для драм-машины и триггери сэмплы на основе эвклидовых массивов.
2. Управляй логикой ML-185: следи за тем, сколько тиков длится текущая стадия, и переключай её.
3. Бери ноту из сетки MDD Snake (на основе текущего индекса

ML-185) и отправляй её в AcidSynth.

Этап 4: Пользовательский Интерфейс (UI)

Промт 11: Навигация и Табы

Реализуй навигацию внутри Mini App. Сделай компонент TabBar внизу экрана с иконами в стиле Telegram iOS. Вкладки: "Drums", "Bass", "Sequencer", "Export". Контент должен переключаться без перезагрузки страницы. Используй CSS-переменные Telegram (var(--tg-theme-bg-color)) для фона активного таба.

Промт 12: UI Компоненты (Knob и Fader)

Создай переиспользуемый компонент Knob (крутилка), который управляется вертикальным свайпом (drag). Он должен показывать текущее значение и поддерживать двойной клик для сброса. Стилизуй его минималистично: кольцо прогресса SVG. Также создай компонент Selector для выбора типа волны или режима (например, алгоритма движения Snake).

Промт 13: Интерфейс Драм-машины

Сверстай вкладку "Drums". Для каждого канала (Kick, Snare, Hats) выведи контролы: Steps(Knob), Pulses (Knob), Shift (Knob). Визуализируй ритм как кольцо или полоску из точек (активные/неактивные шаги), которая обновляется реактивно при изменении параметров эвклидова алгоритма.

Промт 14: Интерфейс Баса (Sting Controls)

Сверстай вкладку "Bass". Размести два больших главных регулятора: Density и Type. Добавь кнопки выбора формы волны (Saw/Square) и ручки для Cutoff и Resonance. Добавь кнопку "Generate New Pattern" (иконка кубика), которая вызывает функцию генерации и обновляет Store.

Промт 15: Интерфейс Секвенсора (Grid & Stages)

Сверстай вкладку "Sequencer".

Верхняя часть: Сетка 4x4 (MDD Snake), где каждая ячейка — это нота.

Подсвечивай текущую активную ячейку во время воспроизведения.

Нижняя часть: 8 вертикальных столбиков (ML-185 Stages). Каждый столбик должен иметь слайдеры для Length и Pulses.

Этап 5: Экспорт и Бэкенд

Промт 16: Генерация MIDI файла

Установи библиотеку midi-writer-js. Напиши функцию exportToMidi, которая берет текущее состояние из Zustand (паттерны ударных, басовую линию, мелодию секвенсора) и генерирует бинарный MIDI-файл (BLOB). Сделай так, чтобы генерировалось 4 или 8 тактов лупа.

Промт 17: Настройка Бэкенда (Node.js Bot)

Теперь переходим к бэкенду. Напиши код простого Telegram бота на **Node.js** с использованием библиотеки telegraf и express. Бот должен иметь эндпоинт POST /api/upload-midi. Этот эндпоинт должен принимать JSON с данными: initData (для валидации юзера) и fileData (Base64 строка MIDI файла).

Промт 18: Валидация и Отправка файла

Реализуй в боте функцию валидации initData от Telegram (проверка HMAC), чтобы убедиться, что запрос пришел от реального пользователя. После валидации декодируй Base64 в буфер и используй метод telegram.sendDocument, чтобы отправить MIDI-файл пользователю в чат с подписью "Ваш MIDI луп готов".

Промт 19: Связь Фронтенда и Бэкенда

На вкладке "Export" во фронтенде создай кнопку "Send to Telegram Chat". При нажатии:

1. Вызывается функция генерации MIDI.
2. Файл конвертируется в Base64.
3. Отправляется POST запрос на твой бэкенд с window.Telegram.WebApp.initData.
4. Показывается лоадер и уведомление об успехе/ошибке

(используй Telegram.WebApp.showAlert).

Этап 6: Полировка

Промт 20: Финальная оптимизация и UX

Проведи финальную доработку.

1. Добавь обработку события Telegram.WebApp.onEvent('viewportChanged'), чтобы интерфейс не перекрывался клавиатурой или системными панелями.
2. Настрой MainButton (главная кнопка Telegram): на вкладке Export она должна запускать отправку файла, на остальных — работать как Play/Stop.
3. Добавь простую визуализацию (осциллограф) на главном экране, подключенную к Tone.Destination.

Конечно. Для успешной реализации такого сложного проекта, как генеративная аудио-станция внутри Telegram, есть ряд критических нюансов, которые не всегда очевидны на старте, но могут стать препятствием при разработке.

Вот дополнительная техническая информация и рекомендации, основанные на особенностях платформы Telegram и Web Audio API.

1. Критическая проблема iOS: "Silent Switch" (Бесшумный режим)

Это самая частая проблема при разработке аудио-приложений для iOS (iPhone). Если у пользователя переключатель сбоку телефона стоит в режиме "Без звука", Web Audio API в браузере (и внутри Telegram WebView) по умолчанию будет молчать, даже если громкость медиа выкручена на максимум.

- **Решение:** Вам нужно "обмануть" систему, заставив её думать, что ваше приложение воспроизводит активный медиа-контент, а не просто звуки интерфейса.
- **Как реализовать:**
  1. При первом взаимодействии пользователя (нажатие кнопки "Start App"), помимо запуска Tone.context.resume(), воспроизведите и тут же остановите короткий пустой HTML5 <audio> тег.
  2. Используйте библиотеку-хелпер, например unmute-ios-audio. Она

автоматически проигрывает тишину в буфере, что переключает аудио-сессию iOS в режим "Playback", игнорирующий рычажок беззвучного режима.

2. Библиотеки для ускорения разработки

Не пишите сложную математику с нуля, если есть проверенные решения. Это сэкономит недели работы.

- **Музыкальная теория:** `tonal` (или `@tonaljs/tonal`).
  - *Зачем:* В ТЗ вы указали выбор тональности. Эта библиотека умеет генерировать массивы нот для любых ладов (Major, Minor, Dorian, Phrygian и т.д.). Вам не нужно хардкодить ноты — вы просто пишете `Scale.get("C minor").notes`, и она возвращает массив, который вы скармливаете генератору баса.
- **Эвклидовы ритмы:** `euclidean-rhythms`.
  - *Зачем:* Реализует алгоритм Бьоркланда. Вызов `getPattern(5, 13)` вернет готовый массив `[1, 0, 0, 1, 0,...]`..
- **Визуальные анимации:** `framer-motion`.
  - *Зачем:* Для React это стандарт де-факто. Она позволит сделать плавные анимации "головы" секвенсора, бегущей по сетке, без просадок FPS, что критично, когда аудио обрабатывается в том же потоке JS.

3. Углубленная логика генератора "Acid" (Sting)

Чтобы генератор баса звучал действительно "кислотно", а не просто рандомно, используйте следующую логику распределения параметров, характерную для жанра:

- **Диапазон нот:** TB-303 имеет диапазон в 3 октавы. Ограничьте генерацию нот диапазоном от **C1 до C3**.
- **Вероятность акцента (Accent):** Хороший грув получается, если акценты падают на слабые доли (off-beat). Попробуйте повышать вероятность акцента для четных шагов (2, 4, 6...).
- **Слайды (Slides):** Слайд не может существовать без ноты. Если генератор поставил слайд на пустой шаг — удаляйте слайд. Слайд всегда должен "тянуть" текущую ноту к *следующей*.
- **Параметр Density (Плотность):** Реализуйте это как "порог прохождения". Присвойте каждому шагу (1-16) случайный "вес" от 0.0 до 1.0 при создании паттерна. Ручка Density (0-100%) просто включает те шаги, чей вес меньше

текущего значения ручки. Это позволит пользователю крутить ручку туда-сюда, добавляя и убирая ноты, но *сохраняя структуру* ритма (ноты не будут прыгать хаотично).

4. Оптимизация производительности (Performance)

Telegram Mini Apps на Android часто работают на бюджетных устройствах, где Web Audio может "заикаться" (glitch).

- **Tone.Draw:** Никогда не обновляйте UI (мигающие лампочки, движение курсора) прямо внутри аудио-колбэка `Tone.Transport.schedule`. Это заблокирует звук.
    - *Как надо:* Используйте `Tone.Draw.schedule(callback, time)`. Это синхронизирует визуализацию с `requestAnimationFrame` браузера, отделяя её от критического аудио-потока.
- **Лимит голосов:** Ограничьте полифонию. `Tone.PolySynth` по умолчанию может плодить много голосов. Установите `maxPolyphony: 6` или меньше для синтезаторов, чтобы не перегрузить CPU.

5. Нюансы Бэкенда и Телеграм Бота

Вы упомянули "историю" и "выгрузку".

- **Хранение истории:** Не храните сгенерированные MIDI файлы на сервере вечно. MIDI весит мало (килобайты), поэтому лучше хранить в базе данных (например, MongoDB или PostgreSQL) **JSON-слепок состояния** (какие ноты где стояли, настройки ручек), а сам MIDI-файл генерировать "на лету" при запросе скачивания. Это сэкономит место и позволит пользователю вернуться и *отредактировать* старый трек из истории, а не просто скачать его.
- **Local Bot API Server:** Если в будущем вы захотите генерировать **Аудио (MP3/WAV)** вместо MIDI, вам придется установить свой локальный сервер для Bot API, так как стандартный сервер Telegram имеет лимит на отправку файлов в 50 МБ (для ботов) и может быть медленным для upload. Для MIDI это не нужно, но держите в уме на будущее.

6. UX Советы для мобильного секвенсора

- **Избегайте "Piano Roll":** На телефоне неудобно тыкать в мелкую сетку пиано-ролла. Используйте подход "Step Sequencer" (как в драм-машинах) или "Scale Mode", где нет "неправильных" нот, а есть просто ступени лада (1, 2, 3...).

- **Double Tap to Reset:** Реализуйте сброс значений ручек (Knobs) в дефолтное положение по двойному тапу. Это стандарт в музыкальном софте, который очень удобен на тачскринах.